SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA5113

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO* (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 TOTAL MARKS: 50

Annexure A
Coding Challenge

Code of Conduct:

I DINDU PUJITHA (192373037) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D PUJITHA

Annexure A
Coding Challenge

1. **Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.**

Answer:

```
import hashlib
import hmac

key = b"AH_secret_key"

ip_packet = b"192.168.1.10 -> 192.168.1.20 : Hello IPSec"

mac = hmac.new(
   key,
   ip_packet,
   hashlib.sha256
).hexdigest()

print("IP Packet:")
print(ip_packet.decode())

print("\nAuthentication Code:")
print(mac)
```

```
received_packet = b"192.168.1.10 -> 192.168.1.20 : Hello IPSec"

received_mac = hmac.new(
    key,
    received_packet,
    hashlib.sha256
).hexdigest()

if hmac.compare_digest(mac, received_mac):
    print("\nAH Verification: SUCCESS")
    print("Packet integrity verified.")
else:
    print("\nAH Verification: FAILED")
    print("Packet has been modified.")

modified_packet = b"192.168.1.10 -> 192.168.1.20 : Modified Data"

modified_mac = hmac.new(
    key,
    modified_packet,
    hashlib.sha256
).hexdigest()

if hmac.compare_digest(mac, modified_mac):
    print("\nModified Packet: VALID")
else:
    print("\nModified Packet: INVALID")
```

```
print("Modification detected.")
```

Analysis

The program:

Creates an IP packet.

Generates an HMAC using a secret key.

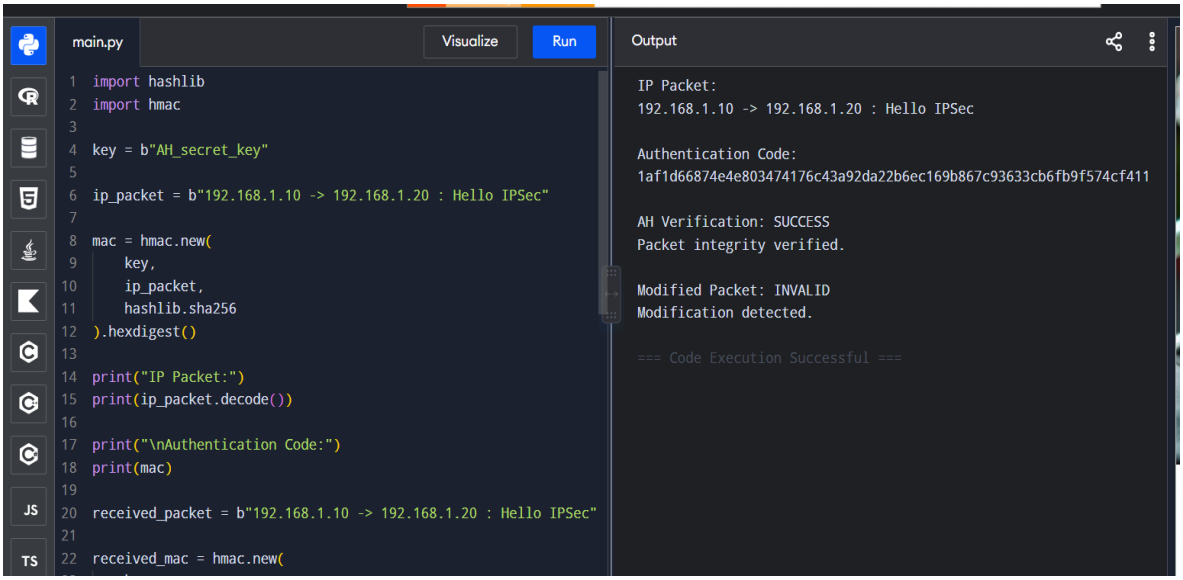
Sends the packet with its authentication code.

Recalculates the MAC at the receiver.

Compares both MAC values.

Detects packet modification.

Conclusion: AH provides **integrity and authentication**, but unlike ESP, it does not provide confidentiality.



2. **Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.**

Answer:

```
key = 123

payload = "This is a confidential IP packet payload."

print("Original Payload:")
print(payload)

encrypted = []

for character in payload:
    encrypted.append(ord(character) ^ key)

print("\nEncrypted Payload:")
print(encrypted)

decrypted = ""

for value in encrypted:
    decrypted += chr(value ^ key)

print("\nDecrypted Payload:")
print(decrypted)
```

```
        if decrypted == payload:
            print("\nESP Verification: SUCCESS")
            print("Payload decrypted successfully.")
        else:
    print("\nESP Verification: FAILED")
```

Analysis

Original Data

↓

Encryption

↓

Encrypted Payload

↓

Transmission

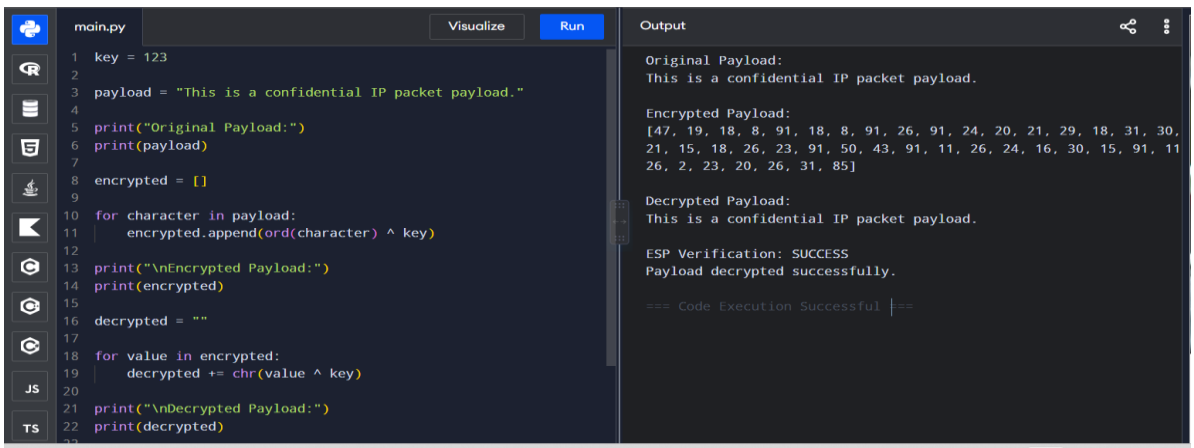
↓

Decryption

↓

Original Data

Conclusion: ESP protects the confidentiality of IP packet payloads. In actual IPSec, secure algorithms such as **AES**



3. **Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.**

   Answer:

```
security_associations = {}


def create_sa(spi, source, destination, algorithm):
    security_associations[spi] = {
        "source": source,
        "destination": destination,
        "algorithm": algorithm
    }

    print("Security Association created.")
    print("SPI:", spi)


def lookup_sa(spi):
    if spi in security_associations:
        print("\nSecurity Association Found:")
        print(security_associations[spi])
    else:
        print("\nSecurity Association not found.")


def delete_sa(spi):
    if spi in security_associations:
        del security_associations[spi]
```

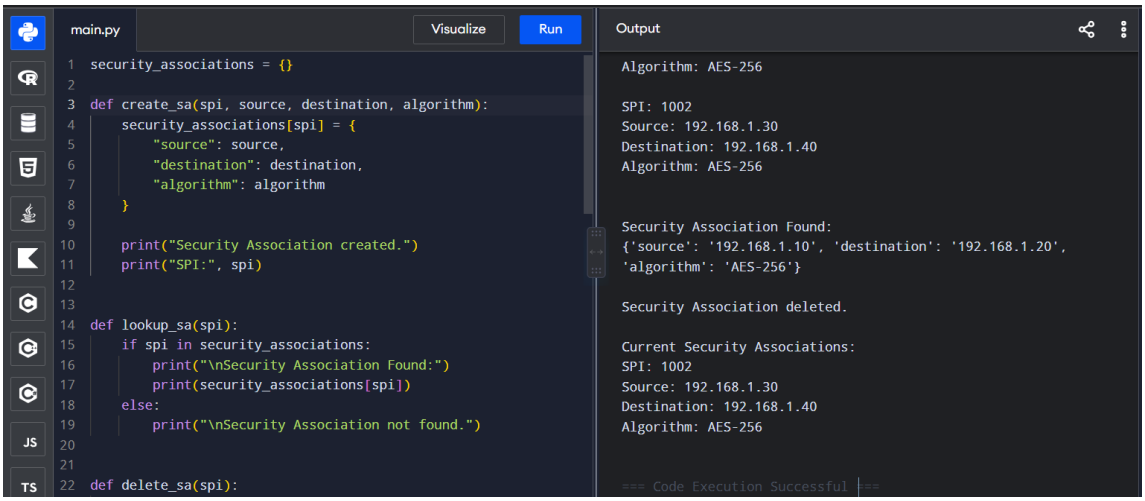
```
        print("\nSecurity Association deleted.")
    else:
        print("\nSecurity Association not found.")


def display_sas():
    print("\nCurrent Security Associations:")

    if not security_associations:
        print("No Security Associations available.")
    else:
        for spi, details in security_associations.items():
            print("SPI:", spi)
            print("Source:", details["source"])
            print("Destination:", details["destination"])
            print("Algorithm:", details["algorithm"])
            print()


create_sa(
    1001,
    "192.168.1.10",
    "192.168.1.20",
    "AES-256"
)

create_sa(
    1002,
```

```
            "192.168.1.30",
            "192.168.1.40",
            "AES-256"
        )

        display_sas()

        lookup_sa(1001)

        delete_sa(1001)

display_sas()
```

Analysis

A Security Association stores security parameters such as:

- **SPI (Security Parameters Index)**
- Source address
- Destination address



4. **Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.**

Answer:

```
import hashlib

p = 61
q = 53

n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = pow(e, -1, phi)

message = "Hello, this is a secure email."

print("Original Email:")
print(message)

hash_value = int(
   hashlib.sha256(message.encode()).hexdigest(),
   16
) % n

signature = pow(hash_value, d, n)

print("\nDigital Signature:")
```

```
print(signature)

key = 23

encrypted = []

for character in message:
    encrypted.append(ord(character) ^ key)

print("\nEncrypted Email:")
print(encrypted)

decrypted = ""

for value in encrypted:
    decrypted += chr(value ^ key)

print("\nDecrypted Email:")
print(decrypted)

received_hash = int(
    hashlib.sha256(decrypted.encode()).hexdigest(),
    16
) % n

verification = pow(signature, e, n)

if verification == received_hash:
```

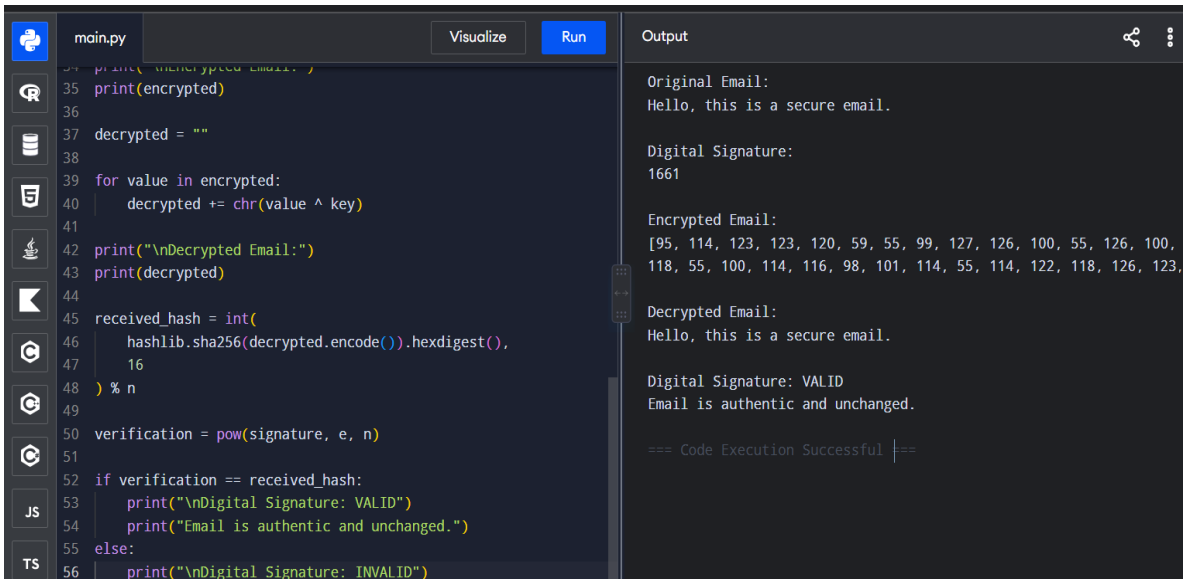
```
            print("\nDigital Signature: VALID")
            print("Email is authentic and unchanged.")
        else:
    print("\nDigital Signature: INVALID")
```

Analysis

A PGP system combines:

- **Encryption** → confidentiality
- **Hashing** → integrity
- **Digital signatures** → authentication

Conclusion: PGP provides secure email communication by combining encryption and digital signatures. This program is a simplified educational model rather than interoperable OpenPGP software.



5. **Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.**

Answer:

```
spam_words = [
    "free",
    "winner",
    "prize",
    "money",
    "offer",
    "urgent",
    "click here",
    "lottery",
    "congratulations"
]

blacklisted_senders = [
    "spam@example.com",
    "winner@example.com"
]



def classify_email(sender, subject, body):
    score = 0
```

```
        if sender.lower() in blacklisted_senders:
            score += 3

        text = (subject + " " + body).lower()

        for word in spam_words:
            if word in text:
                score += 1

        if score >= 3:
            return "SPAM"
        else:
            return "LEGITIMATE"



    sender = input("Enter sender email: ")
    subject = input("Enter email subject: ")
    body = input("Enter email body: ")

    result = classify_email(sender, subject, body)

print("\nEmail Classification:", result)
```

Analysis

The program analyzes:

Email headers:

- Sender address
- Blacklisted sender

Email content:

- Spam-related keywords
- Suspicious phrases

A score is calculated based on detected indicators. If the score reaches a threshold, the email is classified as spam.

Conclusion

Rule-based spam filtering is simple and fast, but it can have difficulty detecting new spam patterns. More advanced s